

Prime Stack Defect Report

Course: Software Validation, Verification and Security Testing
Application under test: Frost Inventory and Shopping System
Team: Prime Stack
Environment: Local backend `http://localhost:3000` , frontend `http://localhost:4200`
Report date: 18 May 2026

1. Defect Origin Summary

Not every defect came from the same test type. DEF-001 and DEF-002 were confirmed by backend functional tests. DEF-005, DEF-006, and DEF-007 come from security test design/scripted security checks. DEF-008 comes from the ZAP scan/header verification. DEF-003 and DEF-004 are source-code review/API robustness defects.

ID	Title	Origin	Severity	Status
DEF-001	Missing/invalid email returns 500 instead of 400	Functional tests TC-FUNC-002, TC-FUNC-008	Medium	Fixed
DEF-002	Template literal bug in <code>getPurchaseByUserId</code>	Functional test TC-FUNC-032	Medium	Fixed
DEF-003	<code>SupplierRoute</code> POST has no error handling	Source-code review	Medium	Fixed
DEF-004	Login route hardcodes status 500	Functional/API error-handling review	Medium	Fixed
DEF-005	Mass assignment allows any user to register as admin	Security test ST-005	Critical	Fixed
DEF-006	<code>/User/users</code> exposes all user data without authentication	Security test ST-007 and route review	High	Fixed
DEF-007	XSS payload stored in database without sanitization	Security test ST-003	Medium	Fixed
DEF-008	Missing security headers	ZAP scan/header verification	Low	Fixed

2. Functional Test Evidence Summary

Before the fixes, the functional suite result was:

```
```text
Test Suites: 2 failed, 2 passed, 4 total
Tests: 3 failed, 33 passed, 36 total
```
```

After fixing the functional and API defects, the functional suite result was:

```
```text
Test Suites: 4 passed, 4 total
Tests: 36 passed, 36 total
```
```

After fixing the security defects, the k6 security result was:

```
```text
checks_succeeded: 100.00% 7 out of 7
checks_failed: 0.00% 0 out of 7
```
```

```
A worker process has failed to exit gracefully and has been force exited. This is likely caused by tests leaking due to improper teardown. Try running with --detectOpenHandles to find leaks. Active timers can also cause this, ensure that .unref() was called on them.
```

```
Test Suites: 2 failed, 2 passed, 4 total
```

```
Tests: 3 failed, 33 passed, 36 total
```

```
Snapshots: 0 total
```

```
Time: 2.399 s
```

```
Ran all test suites.
```

```
Force exiting Jest: Have you considered using `--detectOpenHandles` to detect async operations that kept running after all tests finished?
```

```
ahmet@ahmet-Vivobook-ASUSLaptop-M6500QC:~/WebstormProjects/meanProject/backend$
```

Functional test result before fixes

```
ahmet@ahmet-Vivobook-ASUSLaptop-M6500QC:~/WebstormProjects/meanProject$ cd backend
ahmet@ahmet-Vivobook-ASUSLaptop-M6500QC:~/WebstormProjects/meanProject/backend$ npx jest --config jest.config.json --runInBand --silent
```

```
PASS tests/functionality/purchase.test.js
```

```
PASS tests/functionality/user.test.js
```

```
PASS tests/functionality/product.test.js
```

```
PASS tests/functionality/supplier.test.js
```

```
Test Suites: 4 passed, 4 total
```

```
Tests: 36 passed, 36 total
```

```
Snapshots: 0 total
```

```
Time: 1.578 s, estimated 2 s
```

```
Force exiting Jest: Have you considered using `--detectOpenHandles` to detect async operations that kept running after all tests finished?
```

Functional test result after fixes

DEF-001: Missing/Invalid Email Returns 500 Instead of 400

Origin

Functional testing. Confirmed by:

- TC-FUNC-002: should reject registration with missing email
- TC-FUNC-008: should reject invalid email format

Details

| Field | Value |
|--------------|--|
| Severity | Medium |
| Status | Fixed |
| Test file | backend/tests/functionality/user.test.js |
| Source files | backend/domain/user/valueObjects/Email.js , backend/api/UserRoute.js |

Before the fix, `Email.js` threw plain `Error` objects. `UserRoute.js` checks `error.statusCode` ; plain errors do not have that property, so the route fell back to `500` .

Before:

```
``js
throw new Error("Email required");
```

```
throw new Error("Invalid email format");
...

```

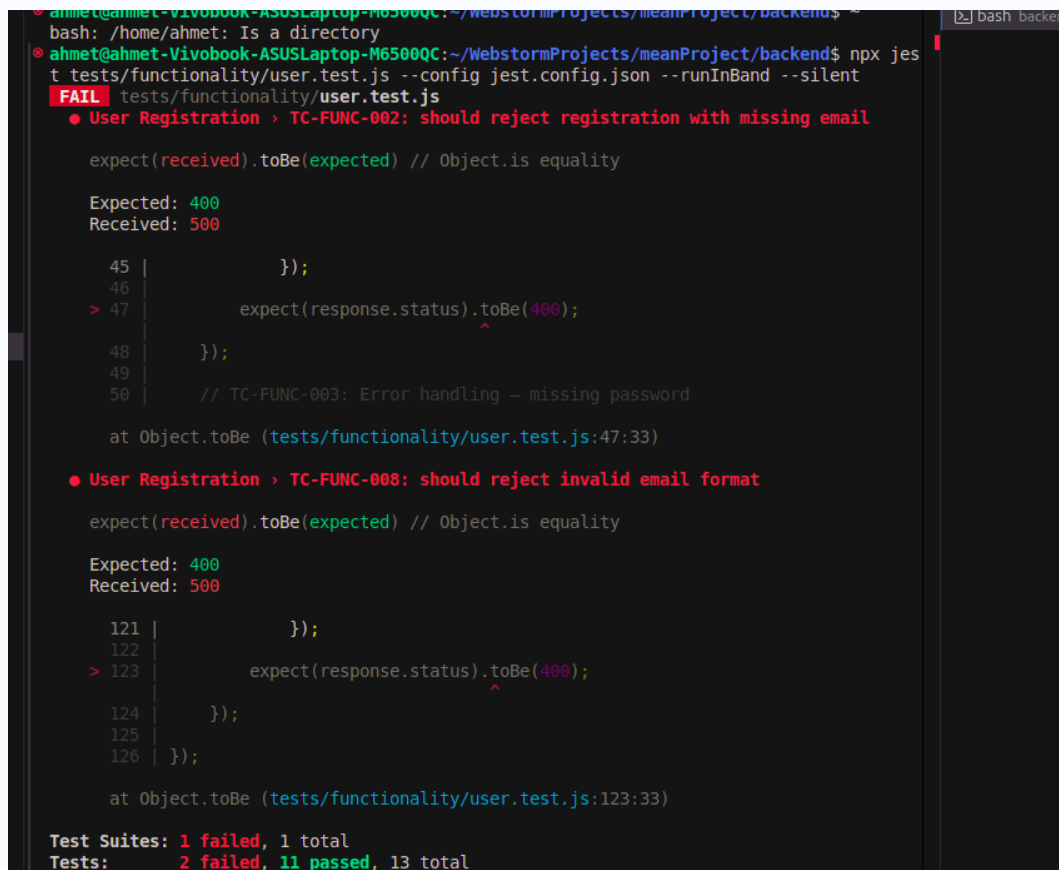
After:

```
``js
throw new ValidationError("Email required");
throw new ValidationError("Invalid email format");
...

```

The route behavior then correctly maps the validation failure to `400 Bad Request`.

Evidence



The screenshot shows a terminal window with the following content:

```
ahmet@ahmet-Vivobook-ASUSLaptop-M6500QC:~/WebstormProjects/meanProject/backend$ npx jest
bash: /home/ahmet: Is a directory
ahmet@ahmet-Vivobook-ASUSLaptop-M6500QC:~/WebstormProjects/meanProject/backend$ npx jest
t tests/functionality/user.test.js --config jest.config.json --runInBand --silent
FAIL tests/functionality/user.test.js
  ● User Registration › TC-FUNC-002: should reject registration with missing email

    expect(received).toBe(expected) // Object.is equality

    Expected: 400
    Received: 500

      45 |         });
      46 |
    > 47 |         expect(response.status).toBe(400);
         |                                   ^
      48 |     });
      49 |
      50 |     // TC-FUNC-003: Error handling – missing password

    at Object.toBe (tests/functionality/user.test.js:47:33)

  ● User Registration › TC-FUNC-008: should reject invalid email format

    expect(received).toBe(expected) // Object.is equality

    Expected: 400
    Received: 500

      121 |         });
      122 |
    > 123 |         expect(response.status).toBe(400);
         |                                   ^
      124 |     });
      125 |
      126 | });

    at Object.toBe (tests/functionality/user.test.js:123:33)

Test Suites: 1 failed, 1 total
Tests:       2 failed, 11 passed, 13 total

```

DEF-001 failing user tests

```

22 router.get('/status', (req, res) => {
43 }
44 });
45
46
47 router.post("/register", async (req, res) => {
48   try {
49     const user = req.body;
50     const result = await userApplication.createUser(user);
51
52     res.status(201).json({
53       message: 'User registered successfully',
54       user: result.toSafeObject()
55     });
56   } catch (error) {
57     console.error('✗ Registration error:', error);
58     const statusCode = error.statusCode || 500;
59
60     res.status(statusCode).json({
61       error: 'Registration failed',
62       message: error.message
63     });
64   }
65 }
66 });
67
68 router.post("/login", async (req, res) => {
69   try {
70
71
72
73

```

DEF-001 route fallback behavior

```

2
3
4 export default class Email{
5   constructor(value) {
6
7     if (!value) {
8       throw new Error("Email required");
9     }
10
11     const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
12
13     if(!emailRegex.test(value)) {
14       throw new Error("Invalid email format");
15     }
16     this._value = value.toLowerCase().trim();
17
18     Object.freeze(this); // makes it immutable
19   }
20
21   get value() {
22     return this._value;
23   }
24
25   equals(otherValue) {
26     return otherValue instanceof Email && this._value === otherValue;
27   }
28 }
29
30
31
32

```

DEF-001 before fix: plain Error in Email value object

```

1  import {ValidationError} from "../../Utilities/errors.js";
2
3
4  export default class Email{
5  constructor(value) {
6
7      if (!value) {
8          throw new ValidationError("Email required");
9      }
10
11      const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
12
13      if(!emailRegex.test(value)) {
14          throw new ValidationError["Invalid email format"];
15      }
16      this._value = value.toLowerCase().trim();
17
18      Object.freeze(this);//makes it immutable
19  }
20
21
22  get value() {
23      return this._value;
24  }
25
26  equals(otherValue) {
27      return otherValue instanceof Email && this._value === otherValue;
28  }
29
30
31
32  }

```

DEF-001 after fix: ValidationError in Email value object

DEF-002: Template Literal Bug in `getPurchaseByUserId`

Origin

Functional testing. Confirmed by:

- TC-FUNC-032: should return purchases for a valid user id

Details

| Field | Value |
|-------------|---|
| Severity | Medium |
| Status | Fixed |
| Test file | backend/tests/functionality/purchase.test.js |
| Source file | backend/domain/purchase/PurchaseRepository.js |

Before the fix, the SQL string used normal quotes, so `${orderClause}` was sent to MySQL literally.

Before:

```

```js
const sql = 'SELECT * FROM purchase WHERE userId = ? ${orderClause}';
```

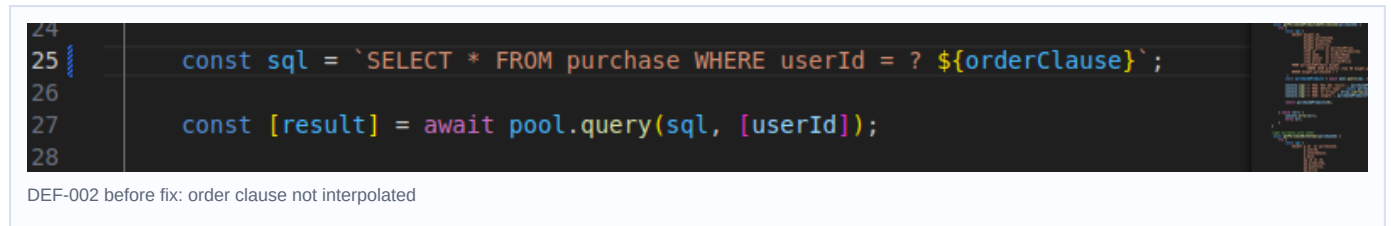
```

After:

```
```js
const sql = `SELECT * FROM purchase WHERE userId = ? ${orderClause}`;
```
```

This allows `ORDER BY date ASC` or `ORDER BY date DESC` to be inserted into the SQL query correctly.

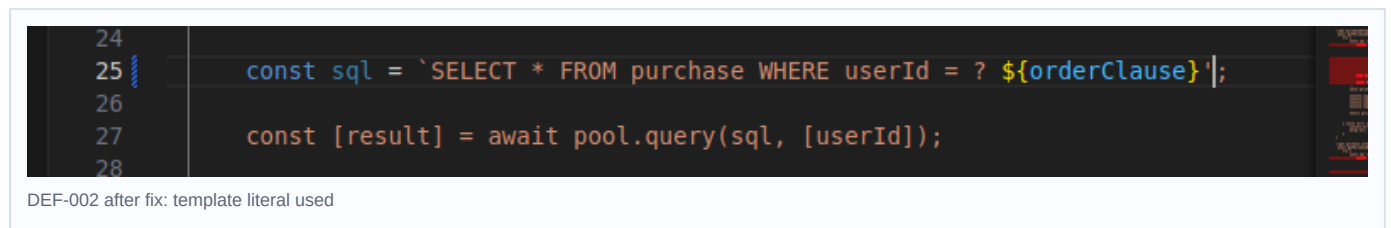
Evidence



DEF-002 before fix: order clause not interpolated

The screenshot shows a code editor with the following code:

```
24
25 const sql = `SELECT * FROM purchase WHERE userId = ? ${orderClause}`;
26
27 const [result] = await pool.query(sql, [userId]);
28
```



DEF-002 after fix: template literal used

The screenshot shows the same code editor with the following code:

```
24
25 const sql = `SELECT * FROM purchase WHERE userId = ? ${orderClause}`;
26
27 const [result] = await pool.query(sql, [userId]);
28
```

DEF-003: `SupplierRoute` POST Has No Error Handling

Origin

Source-code review.

Details

`POST /Supplier/` directly awaits supplier creation without local `try/catch` handling. If validation or duplicate checks fail, the API may return an uncontrolled server error instead of a structured `400` or `409` response.

Fix applied: supplier creation is now wrapped in `try/catch`. The route uses `error.statusCode || 500`, returns a structured JSON error body, and still preserves the previous success behavior.

DEF-004: Login Route Hardcodes Status 500

Origin

Functional/API error-handling review.

Details

The login route returns `500` for invalid login cases. Wrong password, unknown username, and missing credentials should be client/authentication failures.

Expected statuses:

- Missing username/password: `400 Bad Request`
- Wrong username/password: `401 Unauthorized`
- Unexpected backend failure: `500 Internal Server Error`

Fix applied: the login route now uses `error.statusCode || 500`, the same pattern used by the registration route.

Verification:

- Wrong username/password returns `401 Unauthorized` .
- Missing credentials returns `400 Bad Request` .
- Functional suite passed with `36/36` tests.

Evidence

```
    } catch (error) {  
      console.error('✖ Login error:', error);  
      - res.status(500).json({  
      +   const statusCode = error.statusCode || 500;  
      +  
      +   res.status(statusCode).json({  
        error: ' Login failed',  
        message: error.message  
      });  
    }  
  }  
};
```

DEF-004 login route status-code fix

DEF-005: Mass Assignment Allows Any User to Register as Admin

Origin

Security test ST-005.

Details

The security test attempts to register with `role: "admin"` in the request body. Public registration should not accept privileged fields from the client.

Fix applied: public registration no longer trusts `userData.role` . `UserFactory.createUser()` always creates normal users with role `user` ; privileged admin creation must happen through the separate `createAdmin()` path.

Verification from k6:

```
```text  
ST-005 Mass Assignment: 201 ... "role":"user"
✓ ST-005 mass assignment does not grant admin role
```
```

Evidence

```

• ahmet@ahmet-Vivobook-ASUSLaptop-M6500QC:~/WebstormProjects/meanProject$ git diff -- bac
kend/domain/user/UserFactory.js
diff --git a/backend/domain/user/UserFactory.js b/backend/domain/user/UserFactory.js
index 2fff0497..baffb237 100644
--- a/backend/domain/user/UserFactory.js
+++ b/backend/domain/user/UserFactory.js
@@ -16,7 +16,7 @@ export default class UserFactory {
    null,
    username,
    password,
-   userData.role || 'user',
+   'user',
    email,
  );

```

DEF-005 mass-assignment role fix

DEF-006: `/User/users` Exposes All User Data Without Authentication

Origin

Security test ST-007 and route review.

Details

`GET /User/users` should require authentication and admin authorization. Public access can expose user data and support account enumeration.

Expected behavior:

- Unauthenticated request: `401 Unauthorized`
- Authenticated non-admin request: `403 Forbidden`
- Admin request: safe user list only

Fix applied: `/User/users` now uses `requireAdmin`, which returns `401` for unauthenticated users and `403` for users without sufficient privileges.

Verification from k6:

```

```text
ST-007 Unauth User List: 401 - {"error":"Authentication required"}
✓ ST-007 user list requires authentication
```

```

Evidence


```

    });
@@ -177,10 +179,10 @@ router.get('/profile', requireAuth, async (req, res) => {
    });

    //get all users
    -router.get('/users', async (req, res) => {
    +router.get('/users', requireAdmin, async (req, res) => {
        try {
            const allUsers = await userApplication.getAllUsers();
            - res.status(201).json(allUsers)
            + res.status(200).json(allUsers)
        } catch (error) {
            console.error("Failed to fetch all users", error);
            res.status(500).json({
@@ -219,4 +221,4 @@ router.put('/', async (req, res) => {
    });

```

DEF-006 user list admin authorization fix

DEF-007: XSS Payload Stored in Database Without Sanitization

Origin

Security test ST-003.

Details

The security test sends `<script>alert("xss")</script>` as a username. The application should reject or sanitize this input before storage/display.

Fix applied: the `Username` value object now rejects `<` and `>` characters, preventing script-tag usernames from being stored.

Verification from k6:

```

```text

```

```

ST-003 XSS Registration: 400 - {"error":"Registration failed","message":"Username contains invalid characters"}

```

```

✓ ST-003 XSS payload in username is handled

```

```

```

```

Evidence

```

+++ b/backend/domain/user/valueObjects/Username.js
@@ -11,6 +11,10 @@ export default class Username {
    throw new ValidationError("Username must be between 3-50 characters");
  }

  +   if (/[<>]/.test(value)) {
  +     throw new ValidationError("Username contains invalid characters");
  +   }
  +
    this._value = value.trim();
    Object.freeze(this);
  }
@@ -20,4 +24,4 @@ export default class Username {
  }

```

DEF-007 username validation fix

DEF-008: Missing Security Headers

Origin

ZAP scan.

Details

ZAP reported missing or weak security headers:

| Header issue | Risk | Evidence |
|------------------------------------|--------|---|
| CSP missing no-fallback directives | Medium | Content-Security-Policy: default-src 'none' without frame-ancestors and form-action |
| X-Content-Type-Options missing | Low | /Product/ response does not include nosniff |
| X-Powered-By exposed | Low | Response contains X-Powered-By: Express |

Fix applied:

- app.disable('x-powered-by')
- X-Content-Type-Options: nosniff
- Content-Security-Policy: default-src 'self'; frame-ancestors 'none'; form-action 'self'
- Custom JSON 404 handler so missing routes such as /robots.txt no longer fall back to Express's default CSP.

Verification from header check:

```
```text
HTTP/1.1 404 Not Found
X-Content-Type-Options: nosniff
Content-Security-Policy: default-src 'self'; frame-ancestors 'none'; form-action 'self'
```
```

Evidence

```

    /*
+app.disable('x-powered-by');
+
+app.use((req, res, next) => {
+  res.setHeader('X-Content-Type-Options', 'nosniff');
+  res.setHeader('Content-Security-Policy', "default-src 'self'; frame-ancestors 'none'; form-action 'self'");
+  next();
+});
+
+app.use(express.json());
+app.use("/images", express.static(path.join(process.cwd(), "public")));

@@ -87,6 +95,13 @@ app.use('/Purchase',PurchaseRoute);
+app.use('/Uploads',UploadsRoute);
+//app.use(handleMulterError);

+app.use((req, res) => {
+  res.status(404).json({
+    error: 'Not Found',
+    message: `Cannot ${req.method} ${req.path}`
+  });
+});
+

```

DEF-008 security header middleware fix